

Session 8: Time Series Analysis

Compiled by D K. Muriithi

Time Series Analysis: A Comprehensive Guide

1. Introduction to Time Series Analysis

Time series analysis is a statistical technique used to analyze **sequential data points** collected over time. Unlike traditional statistical methods, time series analysis focuses on understanding the structure of data over time, identifying trends, patterns, and seasonality, and making future predictions.

Time series data are commonly found in various fields such as:

- **Finance:** Stock prices, interest rates, exchange rates.
- **Economics:** GDP, inflation rates, unemployment rates.
- **Healthcare:** Patient heart rates, daily COVID-19 cases.
- **Weather and Climate Science:** Temperature, rainfall, wind speed.
- **Business and Marketing:** Sales data, website traffic, customer engagement metrics.

Types of Time Series Data

1. **Univariate Time Series:** A single variable recorded over time, e.g., daily temperature measurements.
2. **Multivariate Time Series:** Multiple related variables recorded over time, e.g., stock prices with trading volume.

Key Characteristics of Time Series Data

1. **Trend:** Long-term increase or decrease in the data.
2. **Seasonality:** Recurring patterns at regular intervals, e.g., higher ice cream sales in summer.
3. **Cyclic Patterns:** Irregular fluctuations that are not fixed in time.
4. **Noise:** Random variations not explained by trends, seasonality, or cycles.

2. Components of Time Series Analysis

Time series data can be decomposed into four key components:

1. Trend (Tt)

A long-term increase or decrease in the data over time.

- Example: An increasing trend in electric vehicle sales over the years.

2. Seasonal Component (St)

Regular and repeating fluctuations in data that occur within a fixed time period (daily, weekly, monthly).

- Example: Increased sales of winter clothing during colder months.

3. Cyclical Component (Ct)

Long-term fluctuations that are **not fixed** in time. These are different from seasonal effects.

- Example: Economic cycles such as recessions and booms.

4. Irregular (Residual) Component (It)

Unpredictable variations caused by unforeseen factors such as natural disasters or sudden economic events.

Decomposition of Time Series

A time series can be expressed as:

- **Additive Model:** ($Y_t = T_t + S_t + C_t + I_t$)
(Used when components remain roughly constant over time.)
 - **Multiplicative Model:** ($Y_t = T_t \times S_t \times C_t \times I_t$)
(Used when seasonal variations change proportionally over time.)
-

3. Time Series Data Preprocessing

Before applying any statistical or machine learning methods, time series data must be cleaned and prepared.

1. Handling Missing Values

- **Interpolation:** Filling missing values using interpolation techniques like linear or spline interpolation.
- **Forward or Backward Fill:** Using the last known value to fill missing data.

- **Mean/Median Imputation:** Replacing missing values with the mean or median.

2. Handling Outliers

Outliers can distort time series analysis. Methods for detecting outliers include:

- **Z-Score Method:** Identifies values that are significantly different from the mean.
- **IQR Method:** Uses the interquartile range to detect extreme values.

3. Transformations

Transformations are used to **stabilize variance and make data more normal-like**.

- **Log Transformation:** Useful for reducing the impact of large values.
- **Differencing:** Helps remove trends and seasonality.

4. Stationarity

A stationary time series has a **constant mean, variance, and autocovariance** over time.

Methods to check for stationarity:

- **Rolling Statistics:** Observing changes in mean and variance.
 - **Augmented Dickey-Fuller (ADF) Test:** A formal statistical test to check stationarity.
-

4. Time Series Forecasting Methods

Several techniques can be used to forecast future values in a time series dataset.

1. Moving Averages

A simple technique that smooths data by averaging values over a fixed window.

Formula for Simple Moving Average (SMA):

$[SMA_t = \frac{1}{n} \sum_{i=0}^{n-1} Y_{t-i}]$ Where:

- (Y_t) is the time series value,
- (n) is the window size.

Simple Moving Average (SMA)

$$SMA_t = \frac{1}{n} \sum_{i=0}^{n-1} Y_{t-i}$$

✅ What it means:

- SMA_t is the average of the last n observations up to time t .
- Y_{t-i} represents the value at time $t - i$.
- It **smooths** the time series to reduce noise and better reveal trends.

2. Exponential Smoothing

A more advanced smoothing technique that assigns exponentially decreasing weights to past observations.

Formula for Simple Exponential Smoothing (SES): $[S_t = \alpha Y_t + (1-\alpha)S_{t-1}]$

Where (α) is the smoothing factor.

- **Holt's Linear Trend Model:** Accounts for trend.
- **Holt-Winters Method:** Accounts for both trend and seasonality.

Simple Exponential Smoothing (SES)

$$S_t = \alpha Y_t + (1 - \alpha)S_{t-1}$$

✅ What it means:

- S_t : Smoothed value (forecast) at time t
- Y_t : Actual observed value at time t
- S_{t-1} : Previous smoothed value
- α : Smoothing constant $(0 < \alpha < 1)$

3. ARIMA (AutoRegressive Integrated Moving Average)

ARIMA is a popular model for forecasting stationary time series data.

$[ARIMA(p, d, q)]$

- **p:** Number of autoregressive (AR) terms.

- **d**: Number of differencing required to make the series stationary.
- **q**: Number of moving average (MA) terms.

To determine **p**, **d**, **q**:

- Use **ACF (Autocorrelation Function)** and **PACF (Partial Autocorrelation Function)**.

4. Seasonal ARIMA (SARIMA)

Used for seasonal time series data: [SARIMA(p, d, q) $\times$ (P, D, Q, m)] Where:

- (P, D, Q) are seasonal parameters,
- (m) is the seasonality period.

SARIMA Notation

$$SARIMA(p, d, q) \times (P, D, Q, m)$$

This model combines both **non-seasonal** and **seasonal** ARIMA components:

Component	Description
p	Order of non-seasonal AR (Auto-Regressive) terms
d	Number of non-seasonal differences
q	Order of non-seasonal MA (Moving Average) terms
P	Order of seasonal AR terms
D	Number of seasonal differences
Q	Order of seasonal MA terms
m	Number of time steps per seasonal cycle (e.g., 12 for monthly data with yearly seasonality)

5. Machine Learning Methods for Time Series

- **Random Forest**: Can capture non-linear relationships but ignores time dependencies.
- **LSTMs (Long Short-Term Memory Networks)**: A type of recurrent neural network (RNN) specifically designed for sequential data.

5. Evaluating Time Series Models

Common metrics to evaluate forecasting models:

- **Mean Absolute Error (MAE):** Measures average absolute differences.
- **Mean Squared Error (MSE):** Penalizes large errors more.
- **Root Mean Squared Error (RMSE):** Square root of MSE.
- **Mean Absolute Percentage Error (MAPE):** Measures percentage error relative to actual values.

Formula for MAE: $\left[\text{MAE} = \frac{1}{n} \sum_{i=1}^n |Y_i - \hat{Y}_i| \right]$

Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |Y_i - \hat{Y}_i|$$

- Measures the **average absolute difference** between actual and predicted values.
- **Less sensitive to outliers** than RMSE.

Formula for RMSE: $\left[\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2} \right]$

Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2}$$

- Measures the **square root of the average squared differences**.
- **More sensitive to large errors/outliers**.

6. Applications of Time Series Analysis

- **Stock Market Prediction:** Identifying trends and making trading decisions.
 - **Sales Forecasting:** Estimating future product demand.
 - **Weather Forecasting:** Predicting temperature, precipitation, and climate changes.
 - **Healthcare Monitoring:** Tracking patient vitals over time.
-

7. Challenges in Time Series Analysis

1. **Non-Stationarity:** Many time series are non-stationary and require transformation.
2. **High Variability:** Fluctuations due to external factors can make forecasting difficult.
3. **Overfitting:** Complex models may capture noise instead of actual patterns.

4. **Computational Complexity:** Large datasets require efficient algorithms.

8. Conclusion

Time series analysis is a powerful tool for understanding trends, patterns, and making predictions based on past data. Traditional statistical models like ARIMA and Holt-Winters remain highly effective, while machine learning approaches, especially deep learning models like LSTMs, are becoming increasingly popular. However, careful preprocessing and model evaluation are crucial for accurate forecasting.

By mastering these techniques, analysts can gain valuable insights and make data-driven decisions across various industries.

Session 8: Time Series Analysis Hands-on Training

```
In [1]: #!pip install --upgrade pip
```

```
In [2]: #!pip install pmdarima
```

```
In [3]: import pandas as pd                                # powerful data mani
import numpy as np                                    # for numerical comp
import matplotlib.pyplot as plt
import seaborn as sns                                # for data analysis
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA         # from pmdarima impor
from sklearn.metrics import mean_absolute_error, mean_squared_error
import warnings
warnings.filterwarnings("ignore")

# Load the data
data = pd.read_csv("murban.csv")
data.head()
```

```
Out[3]:
```

	Months	crude_oil_prices
0	1/1/2003	29.60
1	2/1/2003	31.70
2	3/1/2003	29.10
3	4/1/2003	25.00
4	5/1/2003	25.85

Convert to datetime and set index (Assuming column 'Date' exists)

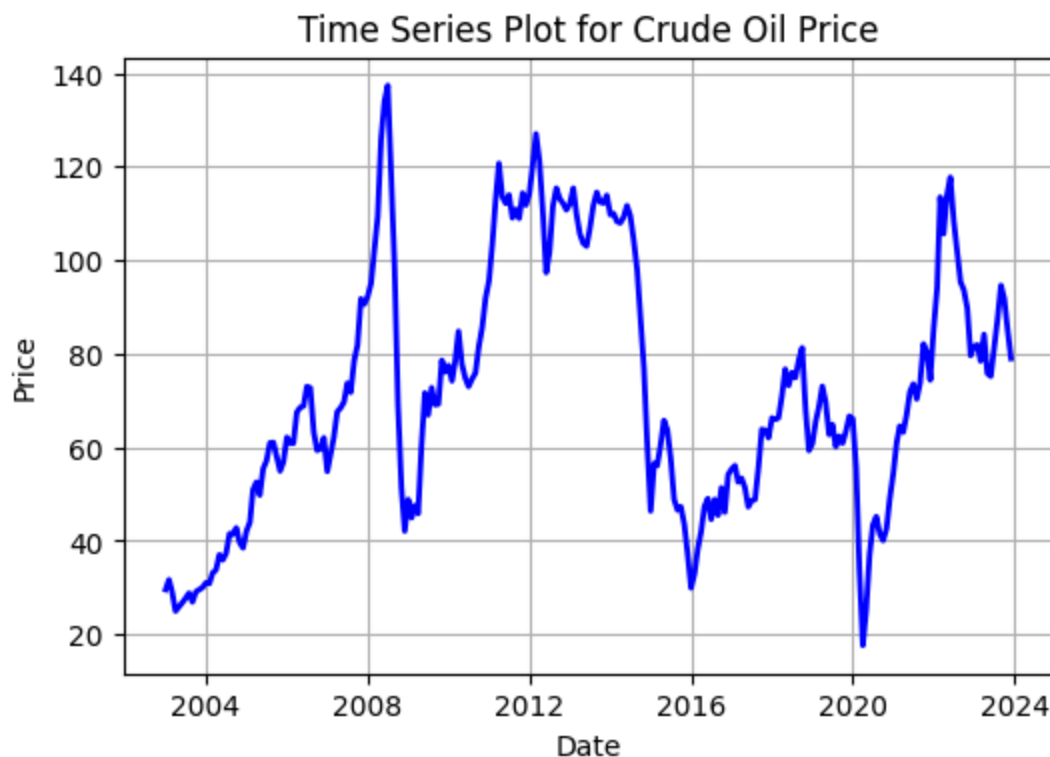
```
In [4]: data['Months'] = pd.to_datetime(data['Months'])
data.set_index('Months', inplace=True)
```

Extract the series of interest

```
In [5]: crude = data['crude_oil_prices']
```

Plot time series

```
In [6]: def plot_series(series, title="Time Series Plot for Crude Oil Price"):
    plt.figure(figsize=(6, 4))
    plt.plot(series, color='blue', linewidth=2)
    plt.title(title)
    plt.xlabel("Date")
    plt.ylabel("Price")
    plt.grid()
    plt.show()
plot_series(crude)
```



Check stationarity (Augmented Dickey-Fuller Test)

```
In [7]: def check_stationarity(series):
    result = adfuller(series)
    print("ADF Statistic:", result[0])
    print("p-value:", result[1])
```



```

if result[1] < 0.05:
    print("The series is stationary.")
else:
    print("The series is non-stationary.")

check_stationarity(crude)

```

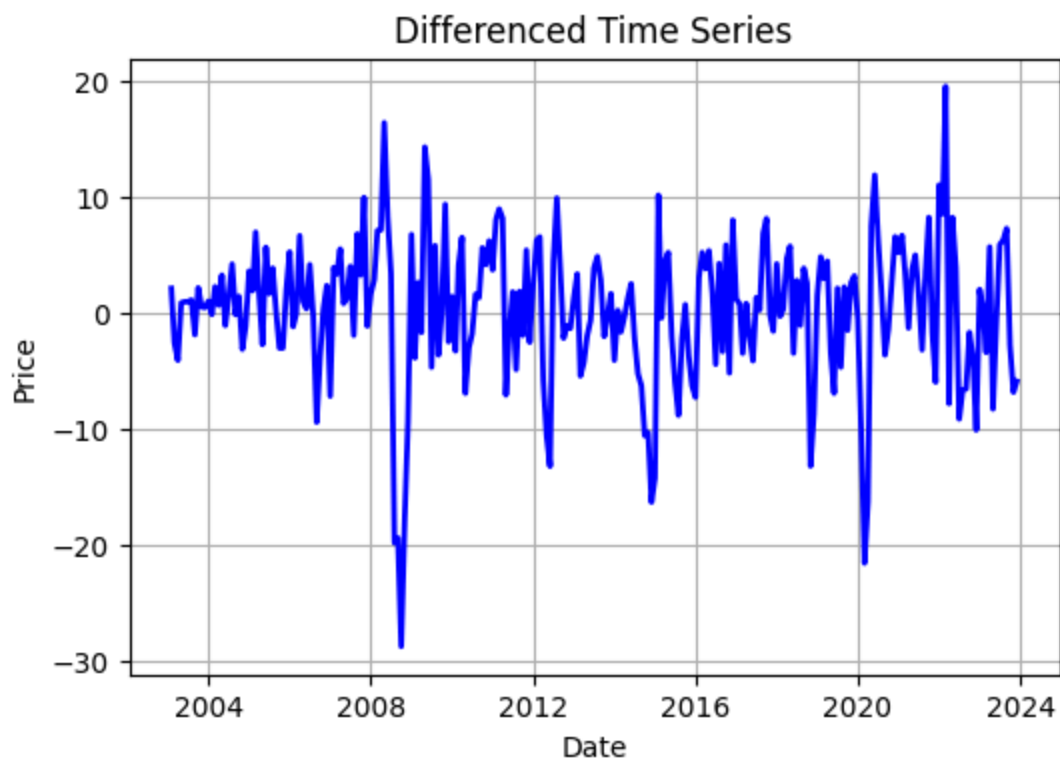
ADF Statistic: -2.824448334341285
p-value: 0.05486546859967542
The series is non-stationary.

Differencing if needed

```

In [8]: crude_diff = crude.diff().dropna()
plot_series(crude_diff, title="Differenced Time Series")
check_stationarity(crude_diff)

```



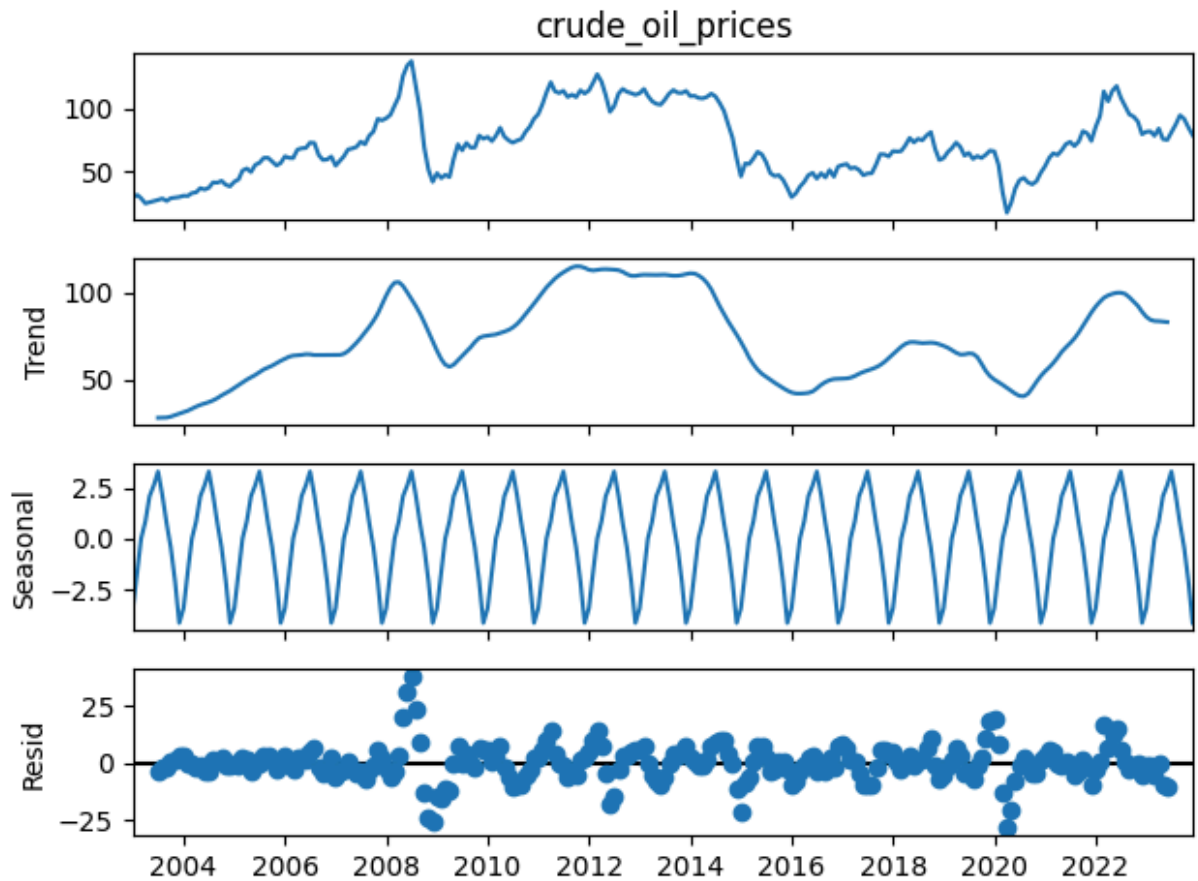
ADF Statistic: -8.934943433942948
p-value: 9.590447049349584e-15
The series is stationary.

Time Series Decomposition

```

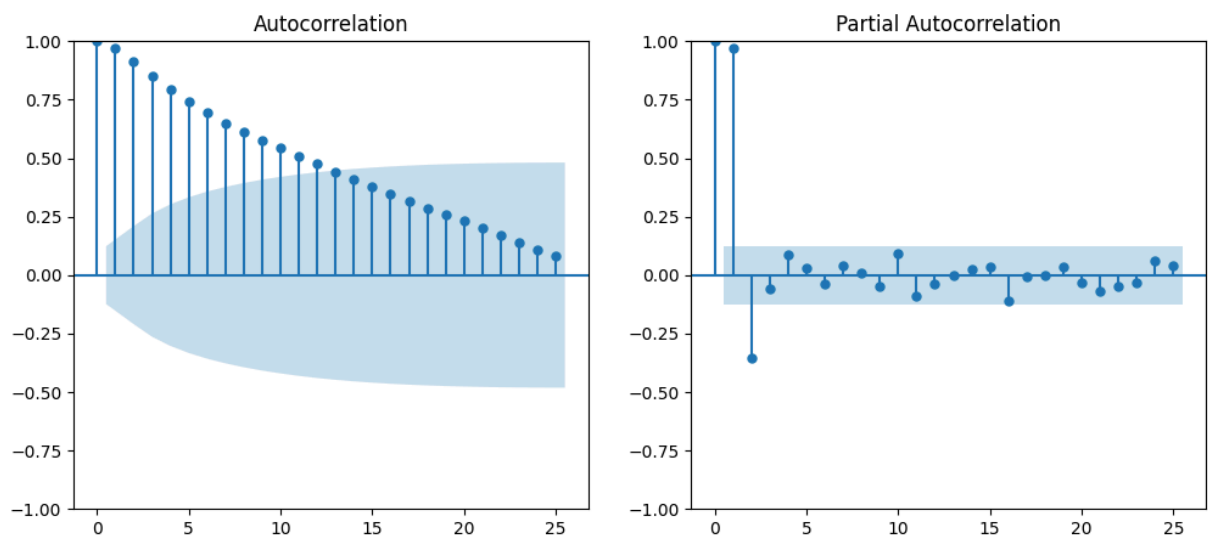
In [9]: decomposition = seasonal_decompose(crude, model='additive', period=12)
decomposition.plot()
plt.show()

```



Plot ACF and PACF

```
In [10]: fig, axes = plt.subplots(1, 2, figsize=(12, 5))
plot_acf(crude, ax=axes[0])
plot_pacf(crude, ax=axes[1])
plt.show()
```



Fit ARIMA model using auto_arima

```
In [11]: #!/pip install pmdarima
```

```
In [12]: # Import the auto_arima function from pmdarima library  
from pmdarima import auto_arima
```

```
model = auto_arima(crude, seasonal=True, m=12, trace=True, suppress_warnings=True)  
print(model.summary())
```

Performing stepwise search to minimize aic

ARIMA(2,1,2)(1,0,1)[12]	intercept	:	AIC=1598.325, Time=1.31 sec
ARIMA(0,1,0)(0,0,0)[12]	intercept	:	AIC=1637.599, Time=0.05 sec
ARIMA(1,1,0)(1,0,0)[12]	intercept	:	AIC=1597.562, Time=0.31 sec
ARIMA(0,1,1)(0,0,1)[12]	intercept	:	AIC=1604.905, Time=0.30 sec
ARIMA(0,1,0)(0,0,0)[12]		:	AIC=1635.846, Time=0.05 sec
ARIMA(1,1,0)(0,0,0)[12]	intercept	:	AIC=1595.941, Time=0.09 sec
ARIMA(1,1,0)(0,0,1)[12]	intercept	:	AIC=1597.471, Time=0.26 sec
ARIMA(1,1,0)(1,0,1)[12]	intercept	:	AIC=1599.020, Time=0.40 sec
ARIMA(2,1,0)(0,0,0)[12]	intercept	:	AIC=1597.815, Time=0.12 sec
ARIMA(1,1,1)(0,0,0)[12]	intercept	:	AIC=1597.871, Time=0.11 sec
ARIMA(0,1,1)(0,0,0)[12]	intercept	:	AIC=1603.178, Time=0.14 sec
ARIMA(2,1,1)(0,0,0)[12]	intercept	:	AIC=1599.256, Time=0.39 sec
ARIMA(1,1,0)(0,0,0)[12]		:	AIC=1594.036, Time=0.07 sec
ARIMA(1,1,0)(1,0,0)[12]		:	AIC=1595.650, Time=0.17 sec
ARIMA(1,1,0)(0,0,1)[12]		:	AIC=1595.557, Time=0.14 sec
ARIMA(1,1,0)(1,0,1)[12]		:	AIC=1597.108, Time=0.25 sec
ARIMA(2,1,0)(0,0,0)[12]		:	AIC=1595.917, Time=0.09 sec
ARIMA(1,1,1)(0,0,0)[12]		:	AIC=1595.969, Time=0.08 sec
ARIMA(0,1,1)(0,0,0)[12]		:	AIC=1601.334, Time=0.08 sec

ARIMA(2,1,1)(0,0,0)[12] : AIC=1597.347, Time=0.23 sec

Best model: ARIMA(1,1,0)(0,0,0)[12]

Total fit time: 4.695 seconds

SARIMAX Results

```
=====
Dep. Variable:          y      No. Observations:      252
Model:                 SARIMAX(1, 1, 0)      Log Likelihood      -795.018
Date:                 Fri, 03 Jul 2026      AIC      1594.036
Time:                 17:09:43      BIC      1601.087
Sample:              01-01-2003      HQIC      1596.873
                   - 12-01-2023
Covariance Type:      opg
=====
```

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	0.4001	0.038	10.499	0.000	0.325	0.475
sigma2	32.9909	2.290	14.409	0.000	28.503	37.478

```
=====
Ljung-Box (L1) (Q):      0.02      Jarque-Bera (JB):      25.85
Prob(Q):                 0.88      Prob(JB):              0.00
Heteroskedasticity (H):  1.11      Skew:                 -0.40
Prob(H) (two-sided):     0.65      Kurtosis:              4.36
=====
```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

Fit and Forecast

```
In [13]: # Import the auto_arima function from pmdarima library
from pmdarima import auto_arima

model = auto_arima(crude, seasonal=True, m=12, trace=True, suppress_warnings=True)
print(model.summary())
```

Performing stepwise search to minimize aic

```
ARIMA(2,1,2)(1,0,1)[12] intercept : AIC=1598.325, Time=1.24 sec
ARIMA(0,1,0)(0,0,0)[12] intercept : AIC=1637.599, Time=0.04 sec
ARIMA(1,1,0)(1,0,0)[12] intercept : AIC=1597.562, Time=0.26 sec
ARIMA(0,1,1)(0,0,1)[12] intercept : AIC=1604.905, Time=0.29 sec
ARIMA(0,1,0)(0,0,0)[12]          : AIC=1635.846, Time=0.05 sec
ARIMA(1,1,0)(0,0,0)[12] intercept : AIC=1595.941, Time=0.09 sec
ARIMA(1,1,0)(0,0,1)[12] intercept : AIC=1597.471, Time=0.26 sec
ARIMA(1,1,0)(1,0,1)[12] intercept : AIC=1599.020, Time=0.40 sec
ARIMA(2,1,0)(0,0,0)[12] intercept : AIC=1597.815, Time=0.12 sec
ARIMA(1,1,1)(0,0,0)[12] intercept : AIC=1597.871, Time=0.10 sec
ARIMA(0,1,1)(0,0,0)[12] intercept : AIC=1603.178, Time=0.15 sec
ARIMA(2,1,1)(0,0,0)[12] intercept : AIC=1599.256, Time=0.32 sec
ARIMA(1,1,0)(0,0,0)[12]          : AIC=1594.036, Time=0.07 sec
ARIMA(1,1,0)(1,0,0)[12]          : AIC=1595.650, Time=0.16 sec
ARIMA(1,1,0)(0,0,1)[12]          : AIC=1595.557, Time=0.13 sec
ARIMA(1,1,0)(1,0,1)[12]          : AIC=1597.108, Time=0.24 sec
ARIMA(2,1,0)(0,0,0)[12]          : AIC=1595.917, Time=0.11 sec
ARIMA(1,1,1)(0,0,0)[12]          : AIC=1595.969, Time=0.09 sec
```

```
ARIMA(0,1,1)(0,0,0)[12] : AIC=1601.334, Time=0.09 sec
ARIMA(2,1,1)(0,0,0)[12] : AIC=1597.347, Time=0.21 sec
```

Best model: ARIMA(1,1,0)(0,0,0)[12]
Total fit time: 4.462 seconds

SARIMAX Results

```
=====
Dep. Variable:          y      No. Observations:      252
Model:                SARIMAX(1, 1, 0)  Log Likelihood    -795.018
Date:                Fri, 03 Jul 2026    AIC              1594.036
Time:                17:09:47    BIC              1601.087
Sample:              01-01-2003    HQIC             1596.873
                  - 12-01-2023
Covariance Type:      opg
=====
              coef    std err          z      P>|z|      [0.025      0.975]
-----
ar.L1          0.4001      0.038     10.499      0.000      0.325      0.475
sigma2        32.9909      2.290     14.409      0.000     28.503     37.478
=====
Ljung-Box (L1) (Q):                0.02  Jarque-Bera (JB):                25.85
Prob(Q):                          0.88  Prob(JB):                      0.00
Heteroskedasticity (H):            1.11  Skew:                          -0.40
Prob(H) (two-sided):              0.65  Kurtosis:                      4.36
=====
```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

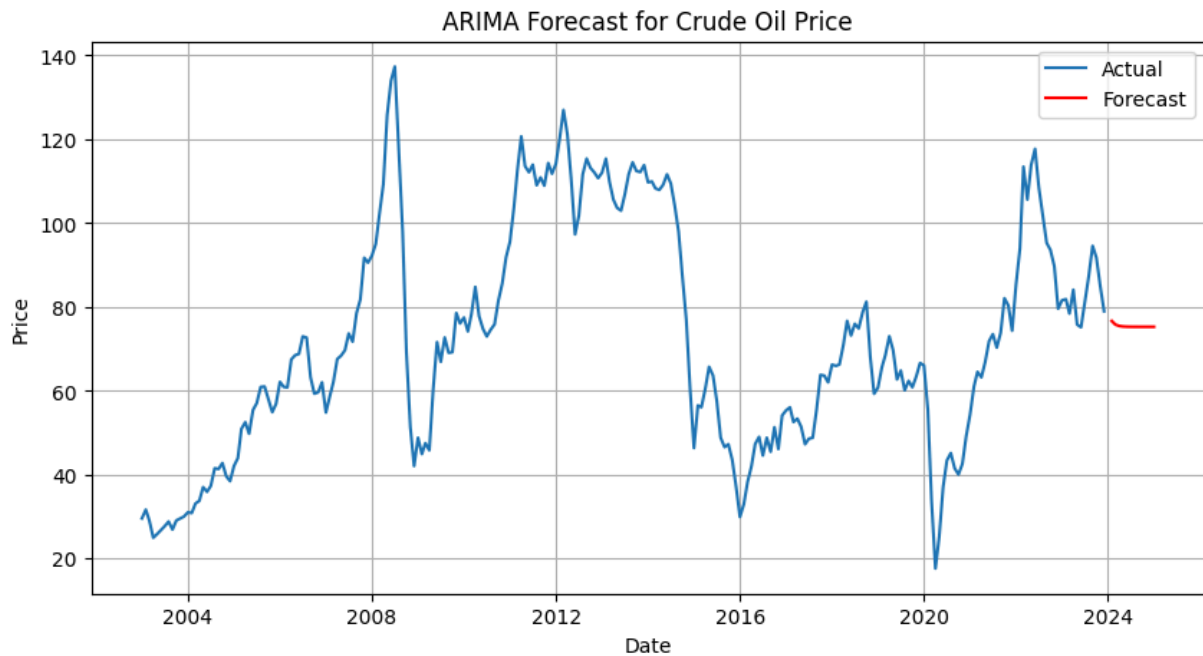
Plot Forecast

```
In [14]: # Import necessary libraries
import matplotlib.pyplot as plt
import pandas as pd
from statsmodels.tsa.arima.model import ARIMA

# Assuming 'crude' is your time series data, first fit an ARIMA model and generate
# You'll need to determine the best (p,d,q) parameters for your specific data
model = ARIMA(crude, order=(1,1,1)) # Example order - adjust based on your data and
fitted_model = model.fit()

# Generate forecast values for the next 12 periods
forecast_values = fitted_model.forecast(steps=12)

# Now create the plot with the defined forecast_values
plt.figure(figsize=(10, 5))
plt.plot(crude, label="Actual")
plt.plot(pd.date_range(crude.index[-1], periods=13, freq='M')[1:], forecast_values,
plt.title("ARIMA Forecast for Crude Oil Price")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.grid()
plt.show()
```



```
In [15]: # Performance Metrics
def evaluate_forecast(actual, predicted):
    mae = mean_absolute_error(actual, predicted)
    mse = mean_squared_error(actual, predicted)
    rmse = np.sqrt(mse)
    print(f"MAE: {mae}, MSE: {mse}, RMSE: {rmse}")

evaluate_forecast(crude[-12:], forecast_values)
```

MAE: 7.519245850631759, MSE: 89.61420111601039, RMSE: 9.46647775658985

<https://cdam.chuka.ac.ke>

Thank you